

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Харківський національний університет імені В.Н. Каразіна

Навчально-науковий інститут комп'ютерних наук та штучного інтелекту

Кафедра математичного моделювання та аналізу даних

ЛАБОЛАТОРНА РОБОТА № 7

на тему: «Робота з тригерами в MSSQL»

Виконав: студент 3 курсу групи кс31
Спеціальності 124 «Комп'ютерні
науки»

Кірієнко Данило

Прийняв: к.т.н., доцент каф. ММ та
АД

Коробчинський К.П.

Мета роботи:

Ознайомитися з поняттям тригерів у системі керування базами даних Microsoft SQL Server, їхнім призначенням, типами, особливостями використання та впливом на цілісність даних. Навчитися створювати, змінювати та видаляти тригери мовою T-SQL, а також використовувати їх для забезпечення бізнес-правил та обмежень цілісності даних.

Завдання 2.

За своїм варіантом (основного, по якому були виконані практичні роботи 3, 4, 5 та 6) додати опис предметної області (скопійовати з варіанту), логічну і фізичну моделі (скопійовати з практичної роботи №5). Наступні запити виконати в зручному для Вас середовищі: Docker, Віртуальна машина з MSSQL, файлова БД або встановлений на локальному комп'ютері екземпляр MSSQL.

Завдання 3.

Створити DML тригер AFTER та INSTED OF й використовувати різні комбінації INSERT, UPDATE, DELETE;

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл TRIGGERS.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 4.

Створити DDL тригер використовувати різні комбінації CREATE, DROP, ALTER; Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл TRIGGERS.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 5.

Створити LOGON тригер;

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;

- Зберегти код запитів додавши у файл TRIGGERS.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 6.

Загалом у створити 3-4 тригери за тематикою своєї лабораторної роботи: які описані у бізнес- правилах та доповнені у практичній роботі 7 і показати яким чином реалізовано кожне з них (у вигляді таблиці):

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл TRIGGERS.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 13.

Завантажити файли до Git-репозиторію та додати посилання у звіт. Репозиторій повинен бути закритим, а доступ відкритий лише для викладачів практики, лабораторних робіт та лектора (k.korobchinskiy@karazin.ua). В репозиторії створити або оновити README.MD файл додавши використання процедур з коротким поясненням зроблених завдань. Використовуючи мову T-SQL зробити скрипти.

- створення таблиць та змінення структури таблиць БД – SETUP.SQL;
- занесення інформації в таблиці БД – INSERT.SQL.
- зміни/оновлення/видалення інформації у таблицях БД – UPDATE.SQL;
- запитів – QUERY.SQL.
- функцій – FUNCTIONS.SQL.
- курсорів – CURSORES.SQL
- тригерів – TRIGGERS.SQL

Всі фали за архівувати та прикипіти до STM.

Завдання 14.

Додати висновки. До оформити звіт та завантажити на STM.

Хід роботи:

Завдання 2.

Вариант №15

Ферма

На ферме имеется несколько участков земли, им присвоены уникальные номера. Каждый участок характеризуется площадью, наличием или отсутствием орошения, посеянной в текущем сезоне культурой. Известна средняя урожайность каждой из возделываемых культур, а также перечень внесенных на каждый участок в данном сезоне удобрений. Для каждой культуры известно: название, урожайность по участкам, нужно ли для нее орошение, какие нужны удобрения. Каждый участок обслуживает одна бригада, но любая бригада может обслуживать более одного участка. Бригада имеет уникальный номер и характеризуется: Ф.И.О. бригадира, Ф.И.О. работников, количеством единиц каждого вида сельхозтехники. О каждом работнике известно, какими видами сельхозмашин (одним, несколькими, ни одним) он может управлять. Работники трудятся на любом из участков бригады, пользуясь техникой, которая есть в бригаде.

Запросы

- ✓ Определить площадь, которая отведена на ферме под каждую культуру с указанием, на каком количестве участков посеяна каждая культура;
- ✓ Вывести всех рабочих заданной бригады с перечнем сельхозтехники, на которой они умеют работать, если такая техника в бригаде имеется, а также определить те виды техники, на которой могут работать члены бригады, но в бригаде такой техники нет, хотя она есть в других бригадах
- для участков данной бригады найти культуру с максимальной урожайностью и проверить, посеяна ли эта культура в текущем сезоне на том участке, где ее урожайность максимальна
- ✓ Найти бригады, которые располагают заданным видом техники в количестве, большем чем среднее количество этой техники по бригадам на всей ферме;
- ✓ Составить список участков (с фамилиями их бригадиров), на которых внесены удобрения, не соответствующие культуре, или не выполняются требования по орошению культуры.

Транзакции

- ✓ Купить новую сельхозтехнику
- ✓ Нанять/уволить сельхозработчих (в том числе и бригадиров).

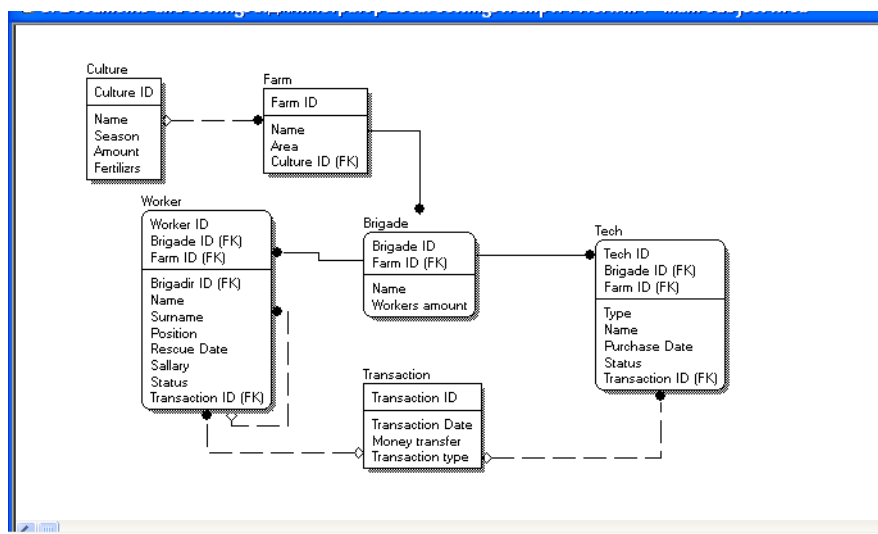


Рисунок 1 - Логічна модель

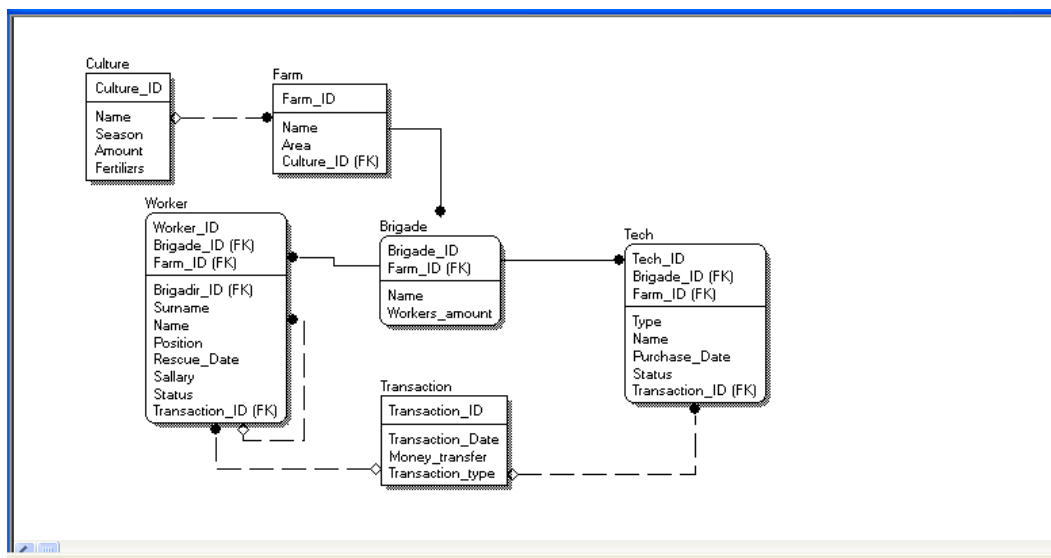


Рисунок 2 - Фізична модель

Завдання 3.

```

CREATE TABLE LogTable (
    Log_ID INT IDENTITY PRIMARY KEY,
    Message NVARCHAR(255),
    CreatedAt DATETIME DEFAULT GETDATE()
);
GO

CREATE TRIGGER trg_AfterInsert_Worker
ON Worker
AFTER INSERT
AS
BEGIN
    INSERT INTO LogTable (Message)
    SELECT 'New worker inserted: ' + Name + ' ' + Surname
    FROM inserted;
END;
GO

CREATE TRIGGER trg_AfterUpdate_TechStatus
ON Tech
AFTER UPDATE
AS
BEGIN
    INSERT INTO LogTable (Message)
    SELECT 'Tech ID ' + CAST(i.Tech_ID AS NVARCHAR) + ' status changed from '
    + d.Status + ' to ' + i.Status
    FROM inserted i
  
```

```

        JOIN deleted d ON i.Tech_ID = d.Tech_ID AND i.Brigade_ID = d.Brigade_ID
AND i.Farm_ID = d.Farm_ID
        WHERE ISNULL(i.Status, '') <> ISNULL(d.Status, '');
END;
GO

CREATE TRIGGER trg_InsteadOfDelete_Transaction
ON [Transaction]
INSTEAD OF DELETE
AS
BEGIN
    UPDATE t
    SET Money_transfer = 0
    FROM [Transaction] t
    JOIN deleted d ON t.Transaction_ID = d.Transaction_ID;
END;
GO

CREATE TRIGGER trg_InsteadOfInsert_Farm
ON Farm
INSTEAD OF INSERT
AS
BEGIN
    INSERT INTO Farm (Farm_ID, Name, Area, Culture_ID)
    SELECT
        Farm_ID,
        Name,
        CASE WHEN Area < 10 THEN 10 ELSE Area END,
        Culture_ID
    FROM inserted;
END;
GO

```

Створили такі тригери:

1. `trg_AfterInsert_Worker` - тригер типу AFTER INSERT, який спрацьовує після додавання нового працівника та записує лог у таблицю `LogTable`.
2. `trg_AfterUpdate_TechStatus` - тригер типу AFTER UPDATE, який фіксує зміну статусу техніки, порівнюючи старе й нове значення, та логує зміну.
3. `trg_InsteadOfDelete_Transaction` - тригер типу INSTEAD OF DELETE, який замість видалення транзакції встановлює суму переказу в 0.

4. `trg_InsteadOfInsert_Farm` - тригер типу `INSTEAD OF INSERT`, який перехоплює вставку ферми та автоматично виправляє площу, якщо вона менша за 10.

Тепер спробуємо додати воркера:

```
INSERT INTO Worker (Worker_ID, Brigade_ID, Farm_ID, Name, Surname,
Position, Rescue_Date, Salary, Status)
VALUES (999, 1, 1, 'Test', 'User', 'Engineer', GETDATE(), 12000, 'Active');

SELECT * FROM LogTable;
```

Results Messages			
	Log_ID	Message	CreatedAt
1	1	New worker inserted: Test User	2025-05-30 13:58:54.537

Рисунок 3 - тригер на додавання робітника

Як бачимо, працює як швейцарський годинник. Далі спробуємо адейтнути статус техніки:

```
UPDATE Tech
SET Status = 'Broken'
WHERE Tech_ID = 1 AND Brigade_ID = 1 AND Farm_ID = 1;

SELECT * FROM LogTable;
```

Results Messages			
	Log_ID	Message	CreatedAt
1	1	New worker inserted: Test User	2025-05-30 13:58:54.537
2	2	Tech ID 1 status changed from Broken to Working	2025-05-30 14:03:41.980
3	3	Tech ID 1 status changed from Working to Broken	2025-05-30 14:03:44.853

Рисунок 4 - лог 2

Додатковий запис виник бо з початку статус вже був брочен, і я вирішив змінити на воркінг. Далі спробуємо видалити транзакцію:

```
SELECT * FROM [Transaction] WHERE Transaction_ID = 1;
DELETE FROM [Transaction] WHERE Transaction_ID = 1;
SELECT * FROM [Transaction] WHERE Transaction_ID = 1;
```

	Transaction_ID	Transaction_Date	Money_transfer	Transaction_type
1	1	2022-11-18	16811.63	Salary

	Transaction_ID	Transaction_Date	Money_transfer	Transaction_type
1	1	2022-11-18	0.00	Salary

Рисунок 5 - підміна при видаленні транзакції

Як і очікувалося, замість видалення воно просто підмінило значення на 0. Далі вставимо ферму, значення area має змінитися на 10:

```
INSERT INTO Farm (Farm_ID, Name, Area, Culture_ID)
VALUES (9999, 'Test Farm', 5, 1);
SELECT * FROM Farm WHERE Farm_ID = 9999;
```

	Fam_ID	Name	Area	Culture_ID
1	9999	Test Farm	10	1

Рисунок 6 - підмінили розмір

Завдання 4.

```
CREATE TABLE DDL_Log (
    Log_ID INT IDENTITY PRIMARY KEY,
    EventType NVARCHAR(100),
    ObjectName NVARCHAR(255),
    SqlCommand NVARCHAR(MAX),
    EventTime DATETIME DEFAULT GETDATE()
);
GO

CREATE TRIGGER trg_DDL_Log
ON DATABASE
FOR CREATE_TABLE, DROP_TABLE, ALTER_TABLE
AS
BEGIN
    DECLARE @EventData XML = EVENTDATA();

    INSERT INTO DDL_Log (EventType, ObjectName, SqlCommand)
    VALUES (
        @EventData.value('(/EVENT_INSTANCE/EventType)[1]',
        'NVARCHAR(100)'),
        @EventData.value('(/EVENT_INSTANCE/ObjectName)[1]',
        'NVARCHAR(255)'),
        @EventData.value('(/EVENT_INSTANCE/TSQLCommand/CommandText)[1]',
        'NVARCHAR(MAX)')
    );
END;
GO
```


Цей тригер відслідковує CREATE, DROP, ALTER. Спробуємо:

```
CREATE TABLE TestDDL (ID INT);
DROP TABLE TestDDL;
ALTER TABLE Farm ADD TestField INT NULL;
```

Results Messages					
	Log_ID	EventType	ObjectName	SqlCommand	EventTime
1	1	CREATE_TABLE	TestDDL	CREATE TABLE TestDDL (ID INT)	2025-05-30 14:19:15.663
2	2	DROP_TABLE	TestDDL	DROP TABLE TestDDL	2025-05-30 14:19:15.667
3	3	ALTER_TABLE	Farm	ALTER TABLE Farm ADD TestField INT NULL	2025-05-30 14:19:15.670

Рисунок 7 - DDL тригер

Завдання 5.

Створимо LOGON тригер:

```
CREATE TABLE Logon_Log (
    Log_ID INT IDENTITY PRIMARY KEY,
    LoginName NVARCHAR(255),
    HostName NVARCHAR(255),
    AppName NVARCHAR(255),
    LogonTime DATETIME DEFAULT GETDATE()
);
GO

CREATE TRIGGER trg_Logon_Audit
ON ALL SERVER
FOR LOGON
AS
BEGIN
    DECLARE @LoginName NVARCHAR(255) = ORIGINAL_LOGIN();
    DECLARE @HostName NVARCHAR(255) = HOST_NAME();
    DECLARE @AppName NVARCHAR(255) = APP_NAME();

    INSERT INTO master.dbo.Logon_Log (LoginName, HostName, AppName)
    VALUES (@LoginName, @HostName, @AppName);
END;
GO
```

Далі спробую вийти і зайти. І тут виникла помилка, я не зміг пройти. Тому прийшлося вмикати локальний сервер з включенимим триггерами, але навіть це вишло не з першого разу. Після довгих спроб все ж таки вийшло видалити тригер, і авжеж, перше що я зроблю, перероблю тригер на безпечний:

```

CREATE TRIGGER trg_Logon_Audit
ON ALL SERVER
FOR LOGON
AS
BEGIN
    BEGIN TRY
        DECLARE @LoginName NVARCHAR(255) = ORIGINAL_LOGIN();
        DECLARE @HostName NVARCHAR(255) = HOST_NAME();
        DECLARE @AppName NVARCHAR(255) = APP_NAME();

        IF @LoginName NOT LIKE 'NT SERVICE\%' AND @LoginName NOT LIKE 'NT
AUTHORITY\%'
        BEGIN
            IF EXISTS (
                SELECT 1 FROM master.sys.tables
                WHERE name = 'Logon_Log' AND SCHEMA_NAME(schema_id) = 'dbo'
            )
            BEGIN
                INSERT INTO master.dbo.Logon_Log (LoginName, HostName,
AppName)
                VALUES (@LoginName, @HostName, @AppName);
            END
        END
    END TRY
    BEGIN CATCH
    END CATCH
END;
GO

```

Results Messages					
	Log_ID	LoginName	HostName	AppName	LogonTime
1	73	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.297
2	72	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.293
3	71	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.290
4	70	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.280
5	69	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.277
6	68	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.273
7	66	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.270
8	67	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.270
9	65	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.133
10	64	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.090
11	63	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.087
12	62	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.083
13	60	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.080
14	61	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.080
15	59	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.077
16	58	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.073
17	57	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.043
18	56	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:19.023
19	55	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:18.880
20	54	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:10:18.870
21	53	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:09:57.977
22	52	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:09:57.973
23	51	MicrosoftAccount\trimarakasa@gmail.com	DANYLO	Microsoft SQL Server Management Studio - Transa...	2025-05-30 15:09:57.967

Рисунок 8 - тригер реєструє входи

Завдання 6.

№	Опис тригера мовою TSQL	Умови запуску / Його тип та опис
1	CREATE TRIGGER TR_Orders_Insert ... AFTER INSERT ON Orders	AFTER INSERT — при додаванні запису в Orders, тригер створює запис в OrdersAudit з дією 'INSERT'
2	CREATE TRIGGER TR_Orders_Update ... AFTER UPDATE ON Orders	AFTER UPDATE — перевірка, що TotalAmount ≤ 10000, інакше ROLLBACK з помилкою. Бізнес-правило на перевищення суми замовлення
3	CREATE TRIGGER TR_Orders_Delete ... INSTEAD OF DELETE ON Orders	INSTEAD OF DELETE — замість видалення запису, змінює поле IsDeleted = 1. М'яке видалення замовлень
4	CREATE TRIGGER TR_DDL_Orders ... ON DATABASE FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE	DDL тригер — лог подій створення/зміни/видалення таблиць у DDLOrdersAudit (аудит дій на рівні структури БД)
5	CREATE TRIGGER trg_Logon_Audit ON ALL SERVER FOR LOGON	LOGON тригер — фіксація входу користувача до БД у таблиці Logon_Log (LoginName, HostName, AppName, LogonTime)

Висновок:

У ході виконання лабораторної роботи було створено п'ять тригерів різних типів: AFTER INSERT, AFTER UPDATE, INSTEAD OF DELETE, DDL та LOGON. Кожен тригер реалізує окреме бізнес-правило, спрямоване на забезпечення цілісності даних, контроль за діями користувачів або логування змін у базі даних. Особливу увагу було приділено безпечному створенню LOGON тригера з обробкою виключень, що дозволяє уникнути блокування доступу. Результати виконання були перевірені на практиці, всі тригери спрацювали відповідно до очікуваної логіки.

GitHub з файлами практичної: <https://github.com/ddanq4/3-obd/tree/main/2%20semestr/lab7>